Лекция 5. Коллективные коммуникации в MPI (начало)

6 октября 2026 г.

Рассмотрим задачу нахождения значения определенного интеграла

$$\int_a^b f(x)dx \tag{1}$$

с использованием численных методов.

Согласно методу трапеций

$$\int_a^b f(x)dx \approx \frac{h}{2}\left(f(a)+f(b)\right)+h\left(f(x_1)+f(x_2)+\dots f(x_{n-1})\right), \tag{2}$$

где n — количество отрезков, на которые делится отрезок $[a,b]$,
$x_0 = a, x_1 = x_0 + h, x_2 = x_0 + 2h, \dots, x_n = b$ — узлы разбиения,
$h = \frac{b-a}{n}$ — шаг разбиения.

Первая попытка решить задачу приведена в листинге 1.

Листинг 1: Вычисление определенного интеграла

```
1  #include <mpi.h>
2  #include <string.h>
3  #include <stdio.h>
4
5  int n;
6  double a;
7  double b;
8
9  int comm_sz;
10 int my_rank;
```

```
11
12    double f(double x)
13    {
14      return x * x;
15    }
16
17    void Input()
18    {
19      if (my_rank == 0)
20      {
21        printf("Enter a, b, n\n");
22        scanf("%lf %lf %d", &a, &b, &n);
23        for (int i = 1; i < comm_sz; i++)
24        {
25          MPI_Send(&a, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
26          MPI_Send(&b, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
27          MPI_Send(&n, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
28        }
29      }
30      else
31      {
32        MPI_Recv(&a, 1, MPI_DOUBLE, 0, 0,
33        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
34        MPI_Recv(&b, 1, MPI_DOUBLE, 0, 0,
35        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
36        MPI_Recv(&n, 1, MPI_INT, 0, 0,
37        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
38      }
39    }
40
41    double Trap(double a, double b, int n, double h)
42    {
43      double res = 0.0;
44      res = 0.5 * (f(a) + f(b));
45      for (int i = 1; i <= n - 1; i++)
46      {
47        double x = a + i * h;
48        res += f(x);
```

```
49      }
50      return res * h;
51    }
52
53    int main()
54    {
55
56      MPI_Init(NULL, NULL);
57
58      MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
59      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
60
61      Input();
62
63      double h = (b - a) / n;
64
65      int local_n = n / comm_sz;
66      double local_a = a + my_rank * local_n * h;
67      double local_b = local_a + local_n * h;
68      double local_int = Trap(local_a, local_b, local_n, h);
69
70      double global_int;
71
72      if (my_rank != 0)
73      {
74        MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
75      }
76      else
77      {
78        global_int = local_int;
79        for (int i = 1; i < comm_sz; i++)
80        {
81          MPI_Recv(&local_int, 1, MPI_DOUBLE, i, 0,
82          MPI_COMM_WORLD, MPI_STATUS_IGNORE);
83          global_int += local_int;
84        }
85      }
86
```

```
87      if (my_rank == 0)
88      printf("Int = %lf\n", global_int);
89
90      MPI_Finalize();
91
92      return 0;
93    }
```

Основной недостаток программы 1 заключается в том, что нахождение глобальной суммы из локальных сумм целиком и полностью выполняется нулевым процессом. Выглядит более разумным использовать древовидную структуру (см. рис. 1). Это позволит снизить время с $O(p)$ до $O(\log(p))$ (p — количество процессов). Реализовать этот алгоритм можно с помощью `MPI_Reduce`:

```
MPI_Reduce(
void*           input_buf_p,        /*in*/
void*           output_buf_p,       /*out*/
int             count,              /*in*/
MPI_Datatype    datatype,           /*in*/
MPI_Op          operation,          /*in*/
int             dest,               /*in*/
MPI_Comm        Communicator        /*in*/
);
```

Функция `MPI_Reduce` реализует **коллективную коммуникацию**. Эта функция берет из каждого процесса значения, на которые указывают `input_buf_p`, и суммирует их, так, чтобы результат оказался в буфере `output_buf_p` процесса `root`. Помимо суммирования, можно выполнять и другие операции, если передать соответствующие аргумент типа `MPI_Op`. Возможно вычисление произведения, нахождение минимума или максимума, логические и побитовые операции. Также можно определить свою собственную операцию.

Примеры других функций, реализующих коллективную коммуникацию

```
MPI_Allreduce(
void*           input_buf_p,        /*in*/
void*           output_buf_p,       /*out*/
int             count,              /*in*/
```

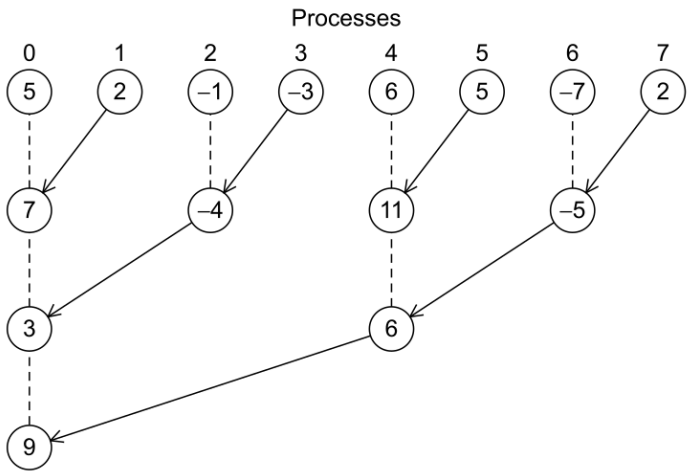


Рис. 1: Нахождение глобальной суммы с помощью `MPI_Reduce`

```
MPI_Datatype    datatype,          /*in*/
MPI_Op          operation,         /*in*/
MPI_Comm        Communicator       /*in*/
);
```

Работает также, как и `MPI_Reduce`, однако полученное значение затем передается всех процессам коммуникатора (см. рис. 2).

```
MPI_Bcast(
void*           input_buf_p,       /*in/out*/
int             count,             /*in*/
MPI_Datatype    datatype,          /*in*/
int             source_proc,       /*in*/
MPI_Comm        Communicator       /*in*/
);
```

Данная функция берет значение, которое хранится в буфере `input_buf_p` на процессе `source_proc`, и отправляет его всем остальным процессам в коммуникаторе, записывая результат в переменную `input_buf_p`.

Работа функции `MPI_Allreduce` сводится к последовательному вызову `MPI_Reduce` и `MPI_Bcast`.

Перепишем программу из листинга 1, используя возможности коллективных коммуникаций. Результат см. в листинге 2

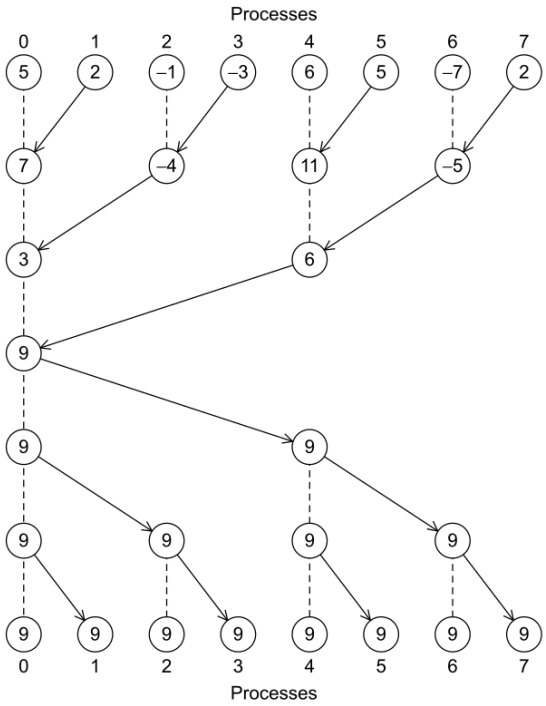


Рис. 2: Механизм работы `MPI_Allreduce`

Листинг 2: Демонстрация силы коллективных коммуникаций

```
1  #include <mpi.h>
2  #include <string.h>
3  #include <stdio.h>
4
5  int n;
6  double a;
7  double b;
8
9  int comm_sz;
10 int my_rank;
11
12 double f(double x)
13 {
14   return x * x;
15 }
16
17 void Input()
18 {
19   if (my_rank == 0)
20   {
21     printf("Enter a, b, n\n");
22     scanf("%lf %lf %d", &a, &b, &n);
23   }
24   MPI_Bcast(&a, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
25   MPI_Bcast(&b, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
26   MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
27 }
28
29 double Trap(double a, double b, int n, double h)
30 {
31   double res = 0.0;
32   res = (f(a) + f(b)) / 2.0;
33   for (int i = 1; i <= n - 1; i++)
34   {
35     double x = a + i * h;
36     res += f(x);
```

```
37      }
38      return h * res;
39    }
40
41    int main()
42    {
43
44      MPI_Init(NULL, NULL);
45
46      MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
47      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
48
49      Input();
50
51      int local_n = n / comm_sz;
52      double h = (b - a) / n;
53
54      double local_a = a + my_rank * local_n * h;
55      double local_b = local_a + local_n * h;
56      double local_int = Trap(local_a, local_b, local_n, h);
57      double global_int = 0.0;
58
59      MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
60      MPI_SUM, 0, MPI_COMM_WORLD);
61
62      if (my_rank == 0)
63      printf("Int = %lf\n", global_int);
64
65      MPI_Finalize();
66
67      return 0;
68    }
```

Свойства функций, относящихся к классу коллективных коммуникаций.

1. Все процессы внутри коммуникатора должны вызывать эту функцию. Если хотя бы один процесс в коммуникаторе не вызовет функцию, то результат будет непредсказуемым.

2. Выходной буфер `output_buffer_p` используется только на процессе с рангом `root`. На других процессах можно передавать что угодно. Так, вызов в строке 59 листинга 2 можно заменить на

```
if(my_rank == 0)
  MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
    MPI_SUM, 0, MPI_COMM_WORLD);
else
  MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
    MPI_SUM, 0, MPI_COMM_WORLD);
```

3. Точечные коммуникации (`MPI_Send, MPI_Recv`) стыкуются на основе совпадения коммуникатора и тега. Коллективные коммуникации не используют тегов, поэтому их стыковка происходит на основе коммуникатора и порядка, в котором вызываются функции. Так, если запустить код из листинга 3 на трех процессах, то значения b и d будут 4 и 5, не 3 и 6.

Листинг 3: Стыковка коллективных коммуникаций

```
int a = 1;
int c = 2;
int b = 0;
int d = 0;

if (my_rank == 0 || my_rank == 2)
{
  MPI_Reduce(&a, &b, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
  MPI_Reduce(&c, &d, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
}
else
{
  MPI_Reduce(&c, &d, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
  MPI_Reduce(&a, &b, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
}

if (my_rank == 0)
printf("b = %d, d = %d\n", b, d);
```